

IDRAAK: From Multi-Agent NLP to Few-Shot Prompting for Semantic Drift Detection in Technical Requirements

Shiva Ahir

Department of Electrical & Computer Engineering
Stony Brook University
New York, United States of America
shiva.ahir@stonybrook.edu

Abstract—Translating technical requirements across languages can introduce semantic drift, altering numerical constraints, polarities, modalities, or other specification-critical meaning. IDRAAK is presented as an interpretable framework for detecting such drift using a language-independent Semantic Requirement Representation (SRR), with six detection workflows evaluated, ranging from deterministic comparison to multi-agent verification and few-shot prompting. On 890 synthetic perturbations across 300 requirements from 10 engineering domains, a single LLM call with six few-shot examples achieves MCC=0.888 and F1=0.983, outperforming the evaluated structured and multi-stage alternatives. Further evaluation on PAWS-X [12] (805 pairs, 5 languages) and XNLI [13] (700 pairs, 7 languages) exposes complementary strengths and limitations of structured and LLM-based approaches. Deterministic SRR comparison performs strongly on technical requirements (F1=0.898) but poorly on general-domain text (F1=0.012), while structured evidence improves performance on adversarial paraphrases. Post-hoc Platt scaling further improves confidence calibration. The results demonstrate that increased agentic complexity does not necessarily improve semantic-drift detection and that simple few-shot prompting can provide a strong and efficient alternative.

✓ Open Source: <https://github.com/shivaahir158/idraak>

Index Terms—semantic drift detection, multilingual technical requirements, multi-agent NLP, few-shot prompting, cross-lingual faithfulness, structured semantic representation, large language models, confidence calibration

I. INTRODUCTION

Technical requirements define precise specifications governing safety-critical systems across aerospace, automotive, medical, and industrial domains [1]. When translated across languages, these requirements risk semantic drift: subtle meaning shifts that alter numerical constraints, invert polarities, weaken modalities, or restructure conditional logic while preserving surface fluency [2]. Unlike general translation errors, such drift can introduce catastrophic specification faults, as when ‘*shall respond within 50 milliseconds*’ becomes ‘*should respond within 50 microseconds*.’

Figure 1 illustrates this problem across languages: a semantically faithful translation preserves the underlying technical constraints despite lexical differences, whereas a

drifting translation may alter numerical values or modality while remaining grammatically fluent.

Existing translation quality metrics are inadequate for this task. BLEU [5] measures surface overlap and is insensitive to meaning-altering substitutions. Neural metrics such as BERTScore [8] and COMET [6] capture semantic similarity but produce opaque scalar scores without identifying what changed or why. Cross-lingual embeddings [30] similarly lack the structured interpretability required for engineering review and certification workflows.

Large language models offer new possibilities for semantic evaluation [15], [16], and multi-agent architectures have been proposed to improve reliability through specialized agent collaboration [19], [21]. However, whether multi-agent complexity improves drift detection over simpler prompting approaches remains empirically unexamined.

This paper presents IDRAAK (*Interpretable Drift Recognition with Agent-Augmented Knowledge*), an interpretable framework for detecting semantic drift in multilingual technical requirements. The contributions are:

- 1) A **Semantic Requirement Representation (SRR)** that decomposes requirements into language-independent typed components including numerical constraints, modalities, conditions, temporal expressions, and domain entities, enabling deterministic field-level comparison across 15 drift categories.
- 2) A **systematic comparison of six detection workflows** spanning deterministic comparison, single-agent hybrid extraction, few-shot direct prompting, ensemble methods combining structured evidence with LLM judgment, full multi-agent verification with eight specialized agents, and back-translation.
- 3) Empirical evidence that a single LLM call with six few-shot examples (MCC=0.888) substantially outperforms the evaluated structured and multi-stage configurations on 890 perturbations across 300 requirements from 10 engineering domains, demonstrating that increased architectural complexity does not necessarily improve drift detection. Further

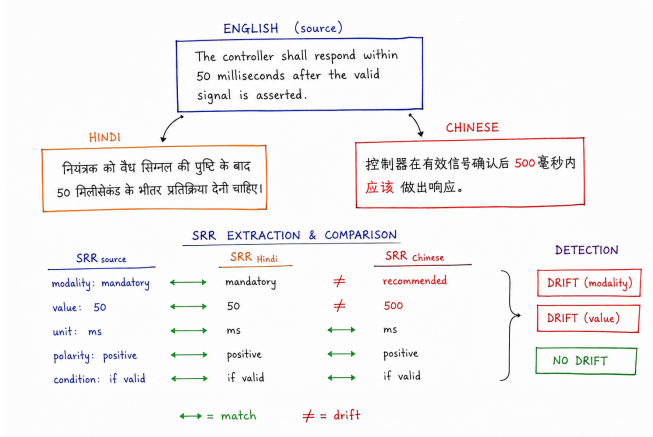


Fig. 1. Cross-lingual semantic drift detection through structured requirement comparison. The Hindi translation preserves the source semantics, while the Chinese translation introduces numerical and modality drift.

validation on PAWSX [12] and XNLI [13] benchmarks, demonstrating cross-domain generalization.

II. RELATED WORK

A. NLP for Requirements Engineering

Natural language processing has been extensively applied to requirements engineering tasks including ambiguity detection, consistency checking, and automated classification [32], [33]. Berry et al. [2] catalogued linguistic sources of ambiguity in software specifications, establishing foundational categories that inform drift taxonomies. However, most work focuses on monolingual English requirements. Cross-lingual requirements analysis remains underexplored, particularly for detecting meaning shifts introduced during translation rather than inherent specification defects.

B. Translation Quality Estimation

Traditional translation evaluation relies on reference-based metrics. BLEU [5] measures n-gram overlap but is insensitive to semantically significant substitutions that share surface similarity. Neural learned metrics including BERTScore [8], COMET [6], and BLEURT [7] leverage contextual embeddings for improved semantic sensitivity, and recent WMT evaluations have demonstrated their superiority over surface metrics [9]. However, these metrics produce scalar similarity scores without structured explanations, making them unsuitable for safety-critical domains where reviewers must understand exactly which technical attributes changed. SRR-based comparison addresses this gap by producing field-level evidence for each detected drift.

C. Semantic Textual Similarity and Paraphrase Detection

Semantic textual similarity (STS) has been studied extensively through shared tasks [10], [11] and dedicated benchmarks. PAWS-X [12] provides adversarial cross-lingual paraphrase pairs where high lexical overlap coexists with different meanings, making it particularly relevant for

evaluating drift detection robustness. XNLI [13] extends natural language inference to 15 languages, enabling cross-lingual evaluation of semantic reasoning. Cross-lingual sentence encoders such as XLM-RoBERTa [30] and multilingual Sentence-BERT [29] provide strong baselines for similarity tasks but, like scalar metrics, lack interpretability at the attribute level. This work differs from standard STS by targeting structured technical content where drift in a single field such as a unit or numerical value constitutes a meaningful semantic change that aggregate similarity scores may miss.

D. Large Language Models as Evaluators

LLMs have demonstrated strong performance as text evaluators when provided with appropriate prompts [16]. Brown et al. [15] established that few-shot prompting enables task adaptation without fine-tuning, and subsequent work has shown that example selection significantly impacts in-context learning effectiveness [23], [24]. Chain-of-thought prompting [17] further improves reasoning on complex tasks. The direct judge workflow builds on these findings, demonstrating that six carefully selected few-shot examples covering representative drift types are sufficient to achieve strong detection performance without task-specific training.

E. Multi-Agent LLM Systems

Multi-agent frameworks decompose complex tasks across specialized LLM-based agents that collaborate through structured interaction. AutoGen [19] enables flexible multi-agent conversations, MetaGPT [20] assigns distinct roles to agents for software development, and multi-agent debate has been shown to improve factuality and reasoning [21], [22]. The underlying hypothesis is that specialized agents with focused responsibilities outperform monolithic approaches. This work provides a controlled empirical test of this hypothesis in the context of semantic drift detection, finding that error propagation across agent boundaries outweighs specialization benefits, and that simpler workflows consistently achieve superior performance.

F. Confidence Calibration

Modern neural networks and LLMs are often poorly calibrated, producing confidence scores that do not reflect true correctness probabilities [25], [28]. Post-hoc calibration methods including Platt scaling [26], isotonic regression [27], and temperature scaling [25] can improve reliability without retraining. These methods are then applied to drift detection confidence scores and demonstrate that Platt scaling reduces expected calibration error from 0.452 to 0.013, an important property for downstream decision-making in safety-critical requirement review.

G. Faithfulness in Text Generation

Related work on faithfulness in abstractive summarization [39] and hallucination detection [40] shares the concern with meaning preservation but operates in a generation context rather than translation verification. Back-translation has been

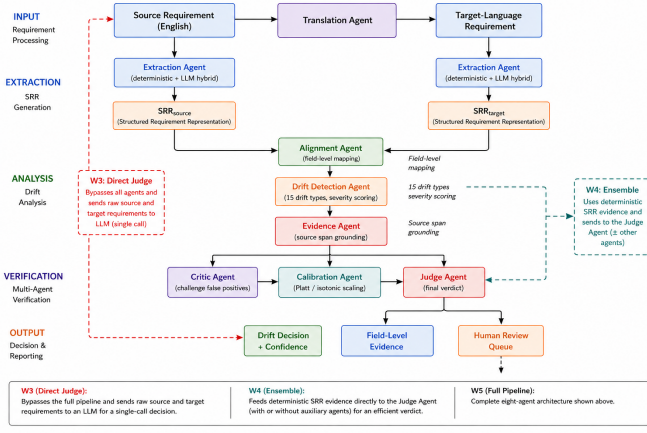


Fig. 2. Architecture of the IDRAAK framework (W5). The system comprises five phases and eight specialized agents for cross-lingual drift detection. W3 and W4 represent simplified workflow variants.

studied as both a data augmentation technique [42] and a quality indicator for translation faithfulness. Back-translation is incorporated as one of the six workflows, enabling round-trip consistency verification as a complementary signal for semantic-drift detection.

III. METHODOLOGY

A. Semantic Requirement Representation

The core of IDRAAK is the Semantic Requirement Representation (SRR), a language-independent structured form that decomposes a technical requirement into typed components. Given a requirement text r , the extraction function $\mathcal{E}(r) \rightarrow S$ produces an SRR S containing:

- **Actor, Action, Object:** the subject performing an action on a target
- **Modality:** obligation level (mandatory, recommended, permitted, optional, forbidden)
- **Polarity:** positive or negative assertion
- **Numerical constraints:** parameter-operator-value-unit tuples (e.g., latency ≤ 50 ms)
- **Conditions:** typed clauses (if, when, unless, only_if) with optional nesting
- **Temporal constraints:** timing relations (before, after, within) with values and reference events
- **Ordering constraints:** sequencing between events (first, second, strict/non-strict)
- **Exceptions:** exclusion clauses (unless, except, provided_that)
- **Entities and Relations:** named domain objects with roles and typed predicates

All components are defined as Pydantic v2 models [41] with strict type validation. The representation is designed so that two requirements with identical meaning in different languages produce structurally equivalent SRRs.

B. SRR Extraction

Three extraction strategies are implemented. **Deterministic extraction** uses regular expressions and pattern matching to parse modality markers, numerical values with units, conditional keywords, temporal expressions, and polarity indicators. It is fast and reproducible but limited to patterns explicitly encoded. **LLM extraction** prompts an LLM to produce a structured SRR as JSON, capturing semantic content that resists pattern matching. **Hybrid extraction** runs both methods and merges results using a priority strategy where deterministic values always override LLM outputs for comparison-critical fields (numerical constraints, modality, polarity, units), while LLM outputs fill fields where deterministic extraction yields nothing (entities, relations, qualifiers). This merge strategy proved critical: without it, LLM extraction introduced noise into fields where deterministic parsing was already accurate.

C. Drift Detection

Given a source requirement r_s and candidate translation r_c , drift detection proceeds by extracting SRRs $S_s = \mathcal{E}(r_s)$ and $S_c = \mathcal{E}(r_c)$, then computing field-level differences. The comparison engine performs unit-aware numerical matching (e.g., recognizing that 1 kB = 1024 bytes), operator compatibility checks, modality ordering comparison, polarity alignment, and set-based matching for entities and conditions. Each difference is classified into one of 15 drift types: numerical, unit, polarity, modality, condition, temporal, threshold, entity, relation, exception, omission, addition, terminology, scope, and reference. Differences are assigned severity levels (none, low, medium, high, critical) and aggregated into a binary drift decision with an associated confidence score.

D. Detection Workflows

Figure 2 provides an overview of the IDRAAK architecture and the relationship among the evaluated workflows. The complete W5 configuration executes the full multi-agent pipeline, while W3 and W4 bypass portions of this pipeline to evaluate whether simpler decision mechanisms can achieve comparable or superior semantic-drift detection. Six workflows of increasing complexity are implemented:

(W1) Deterministic comparison. Extracts SRRs using regex-based parsing and compares fields deterministically. Produces interpretable field-level evidence but is limited to patterns present in the extraction rules.

(W2) Structured single-agent. Uses hybrid extraction (deterministic + LLM) followed by the same field-level comparison. The LLM enriches extraction coverage while deterministic priority preserves precision on critical fields.

(W3) Direct judge. A single LLM call receives both requirement texts along with six few-shot examples [15], [23] covering representative drift types (paraphrase, numerical drift, polarity inversion, entity swap, modality shift, and omission) and returns a binary drift decision with confidence and explanation.

(W4) Ensemble. Runs deterministic SRR extraction and comparison to produce structured evidence, then passes this evidence alongside the original texts to an LLM judge. The LLM makes the final decision informed by both the raw texts and the structured differences.

(W5) Full multi-agent IDRAAK. Eight specialized agents execute sequentially: a *translation agent* handles language conversion, an *extraction agent* produces SRRs, an *alignment agent* maps corresponding fields, a *drift detection agent* identifies differences, an *evidence agent* grounds findings in source text spans, a *critic agent* challenges potential false positives, a *calibration agent* adjusts confidence scores, and a *judge agent* renders the final verdict. Each agent operates with a deterministic fallback when LLM calls fail.

(W6) Back-translation. Translates the candidate back to the source language and compares the back-translated text against the original using SRR comparison [42], providing a round-trip consistency signal.

E. Confidence Calibration

Raw confidence scores from LLM-based workflows are typically poorly calibrated [25], [28]. Post-hoc calibration is performed using Platt scaling [26], which fits a logistic regression on the log-odds of raw confidence scores, and isotonic regression [27], which learns a monotonic mapping from raw scores to calibrated probabilities. Both methods are fitted on validation data and evaluated using Expected Calibration Error (ECE).

F. Evaluation Metrics

Accuracy, precision, recall, and F1 score are reported for completeness, while Matthews Correlation Coefficient (MCC) [35] is used as the primary evaluation metric. MCC accounts for all four quadrants of the confusion matrix and is robust to class imbalance [36], making it more informative than F1 for this task, where the drift-positive class dominates. AUROC and ECE are additionally reported to assess discrimination and calibration quality, respectively.

IV. EXPERIMENTS

A. Datasets

1) *Synthetic Benchmark:* A total of 300 technical requirements are generated and uniformly distributed across 10 engineering domains: digital hardware, embedded systems, software systems, networking, cybersecurity, safety-critical systems, data processing, financial systems, healthcare devices, and industrial automation (30 per domain). Each requirement is generated from domain-specific templates with randomized actors, actions, signals, numerical values, units, modalities, conditions, and temporal constraints. A controlled perturbation engine subsequently produces labeled variants for each requirement, yielding 890 total samples: 142 semantically equivalent paraphrases (label 0) and 748 drifted variants (label 1) spanning 11 drift types. The resulting distribution is presented in Table I. All random seeds are fixed to ensure reproducibility.

TABLE I
DISTRIBUTION OF PERTURBATION TYPES IN THE SYNTHETIC BENCHMARK.

Drift Type	Count
Polarity drift	189
Modality drift	179
Numerical drift	104
Unit drift	89
Threshold drift	53
Temporal drift	36
Scope drift	24
Terminology drift	23
Condition drift	21
Omission drift	19
Entity drift	11
Paraphrase (no drift)	142
Total	890

2) *PAWSX:* Evaluation is conducted on PAWS-X [12], a cross-lingual adversarial paraphrase identification benchmark containing sentence pairs with high lexical overlap but potentially different meanings. Test splits across five languages (en, de, es, fr, zh) are used, totaling 805 pairs (486 paraphrase and 319 non-paraphrase). Paraphrase pairs are mapped to no-drift (label 0), while non-paraphrase pairs are mapped to drift (label 1). This benchmark assesses robustness to adversarial lexical overlap, a challenging scenario in which surface similarity is deliberately misleading.

3) *XNLI:* Evaluation is conducted on XNLI [13], a cross-lingual natural language inference benchmark, using 100 premise-hypothesis pairs per language across seven languages (en, hi, ar, zh, de, fr, es), totaling 700 pairs. Entailment pairs are mapped to no-drift, while contradiction and neutral pairs are mapped to drift. This mapping is imperfect because entailment represents a directional relation, whereas semantic equivalence is symmetric; nevertheless, it provides a useful stress test on non-technical, general-domain text.

B. Experimental Setup

All LLM-based experiments use GPT-4o-mini via the OpenAI API. We evaluate six workflow configurations:

- 1) **Structured Single / Deterministic** (W1): regex-based SRR extraction with field-level comparison
- 2) **Structured Single / OpenAI** (W2): hybrid extraction (deterministic + LLM) with field-level comparison
- 3) **Direct Judge / OpenAI** (W3): single LLM call with six few-shot examples
- 4) **Ensemble / OpenAI** (W4): deterministic SRR evidence passed to LLM judge
- 5) **Full IDRAAK / Deterministic** (W5a): eight-agent pipeline with deterministic fallback
- 6) **Full IDRAAK / OpenAI** (W5b): eight-agent pipeline with LLM-powered agents

The few-shot examples for W3 were manually selected to cover six representative cases: a semantically equivalent

TABLE II
RESULTS ON THE SYNTHETIC BENCHMARK (890 PERTURBATIONS, 300 REQUIREMENTS, 10 DOMAINS).

Workflow	Provider	F1	MCC	Time
Structured Single	Deterministic	0.898	0.474	0.1s
Structured Single	OpenAI	0.918	0.501	181min
Direct Judge	OpenAI	0.983	0.888	17min
Ensemble	OpenAI	0.926	0.411	27min
Full IDRAAK	Deterministic	0.898	0.474	0.2s

paraphrase, a numerical value change, a polarity inversion, an entity substitution, a modality weakening, and an omission. Temperature is set to 0 for all LLM calls to ensure deterministic outputs. Each experiment is run on the full dataset without subsampling.

C. Results

1) *Synthetic Benchmark*: Table II presents the results on the 890-sample synthetic benchmark, while Figure 3 provides a visual comparison of performance across the evaluated workflows. The direct judge (W3) achieves the strongest overall performance, with an F1 score of 0.983 and an MCC of 0.888, substantially outperforming the other approaches. The structured single-agent workflow with hybrid extraction (W2) achieves an MCC of 0.501, indicating that prioritizing deterministic extraction for comparison-critical fields improves the reliability of the hybrid strategy. Both deterministic baselines (W1 and W5a) obtain identical performance (MCC=0.474), suggesting that decomposing the same deterministic operations across multiple agents provides no additional benefit. The ensemble workflow (W4) achieves an F1 score of 0.926 but a lower MCC of 0.411; however, this result should be interpreted with caution because API rate limiting caused approximately 200 samples to fall back to deterministic processing. Overall, Figure 3 highlights that the comparatively simple few-shot direct judge achieves the best performance despite requiring substantially less architectural complexity than the full multi-agent framework.

2) *PAWSX*: Table III presents results on the PAWSX adversarial benchmark. The ensemble workflow achieves the highest MCC=0.459, slightly outperforming the direct judge (MCC=0.451). This reversal from the synthetic benchmark suggests that structured SRR evidence provides a useful inductive bias when the LLM must distinguish adversarial pairs with high lexical overlap. Deterministic comparison fails almost completely (F1=0.012) because general-domain sentences lack the technical patterns (modalities, numerical constraints, units) that drive SRR-based detection.

3) *XNLI*: Table IV presents results on XNLI. All methods achieve low MCC (0.074–0.101), reflecting the fundamental mismatch between entailment and semantic drift. LLM-based workflows predict drift for nearly all pairs (343/700 false positives for the direct judge, zero false negatives), indicating that the model recognizes non-equivalence in premise-hypothesis pairs but cannot distinguish directional

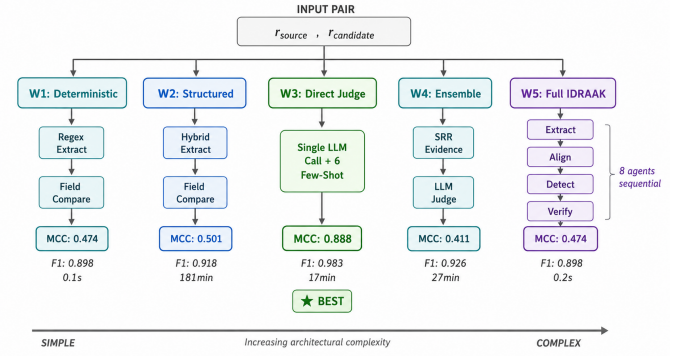


Fig. 3. Performance comparison of the five drift-detection workflows. W3 achieves the best overall performance despite its simpler architecture.

TABLE III
RESULTS ON PAWSX ADVERSARIAL PARAPHRASE BENCHMARK (805 PAIRS, 5 LANGUAGES).

Workflow	Provider	F1	Acc.	MCC
Structured Single	Deterministic	0.012	0.388	−0.098
Direct Judge	OpenAI	0.814	0.739	0.451
Ensemble	OpenAI	0.817	0.738	0.459
Structured Single	OpenAI	0.723	0.583	−0.003
Full IDRAAK	Deterministic	0.012	0.388	−0.098

entailment from bilateral drift. Per-language analysis shows consistent performance across all seven languages with no significant degradation for any individual language.

4) *Confidence Calibration*: Table V reports calibration results on XNLI. Raw LLM confidence scores are poorly calibrated across all workflows, with ECE ranging from 0.120 to 0.464. Platt scaling [26] reduces ECE to below 0.02 in all cases, and isotonic regression [27] achieves near-perfect calibration. These results confirm that post-hoc calibration is essential for reliable confidence scores in drift detection, particularly when confidence thresholds are used for human review routing in safety-critical workflows.

D. Analysis

Per-drift-type detection. Figure 4 shows detection rates across 11 drift types. The direct judge maintains >0.91 accuracy on all types. Deterministic extraction struggles with semantic categories (entity: 0.55, terminology: 0.60, omission: 0.65) but excels on structural ones (polarity: 0.95, modality: 0.92).

Cross-lingual consistency. Figure 5 shows per-language MCC on XNLI. LLM-based workflows exhibit uniform performance across all seven languages (MCC variance <0.1). Deterministic extraction only succeeds on English and Spanish, where regex patterns directly match.

Calibration. Figure 6 shows reliability diagrams before and after Platt scaling. Raw LLM confidence is severely overconfident (ECE=0.452). Platt scaling corrects this to ECE=0.013, aligning predicted confidence with actual accuracy.

TABLE IV
RESULTS ON XNLI BENCHMARK (700 PAIRS, 7 LANGUAGES).

Workflow	Provider	F1	Acc.	MCC
Structured Single	Deterministic	0.396	0.533	0.074
Direct Judge	OpenAI	0.671	0.510	0.101
Ensemble	OpenAI	0.670	0.507	0.085
Structured Single	OpenAI	0.669	0.520	0.091
Full IDRAAK	Deterministic	0.396	0.533	0.074

TABLE V
CALIBRATION RESULTS ON XNLI (EXPECTED CALIBRATION ERROR).

Workflow	Raw ECE	Platt	Isotonic
Direct Judge / OpenAI	0.452	0.013	0.000
Ensemble / OpenAI	0.464	0.001	0.000
Structured Single / OpenAI	0.225	0.018	0.000
Structured Single / Det.	0.120	0.002	0.000
Full IDRAAK / Det.	0.285	0.003	0.000

Cost-performance tradeoff. Figure 7 plots F1 against computation time. The direct judge is Pareto optimal: highest F1 (0.983) at 17 minutes, compared to 181 minutes for structured single with lower F1 (0.918). Deterministic baselines are instantaneous but plateau at F1=0.898.

V. DISCUSSION

A. Why Does the Simpler Few-Shot Approach Perform Best?

A single LLM call with six few-shot examples (MCC=0.888) substantially outperforms the evaluated structured and multi-stage configurations. Sequential decomposition can propagate upstream errors, while intermediate representations may discard contextual cues available during holistic comparison. Well-selected examples also directly encode the distinction between semantic drift and valid paraphrase. These findings suggest that increased architectural complexity does not necessarily improve performance for well-defined semantic classification tasks.

B. When Does Structured Evidence Help?

The ensemble slightly outperforms the direct judge on PAWSX (MCC=0.459 vs. 0.451), despite underperforming on the synthetic benchmark. Structured SRR evidence therefore appears useful for adversarial inputs where lexical similarity is misleading [12]. Technical drift is often exposed through values, units, modalities, and conditions, suggesting that structured evidence can complement holistic LLM reasoning.

C. Domain Specificity as Strength and Limitation

Deterministic SRR comparison achieves F1=0.898 on technical requirements but only F1=0.012 on PAWSX. SRR is intentionally specialized for technical constructs such as numerical constraints, units, modalities, conditions, and temporal relations. General-domain sentences often lack these structures, producing sparse representations. Domain

Detection Rate by Drift Type
(Synthetic Benchmark)

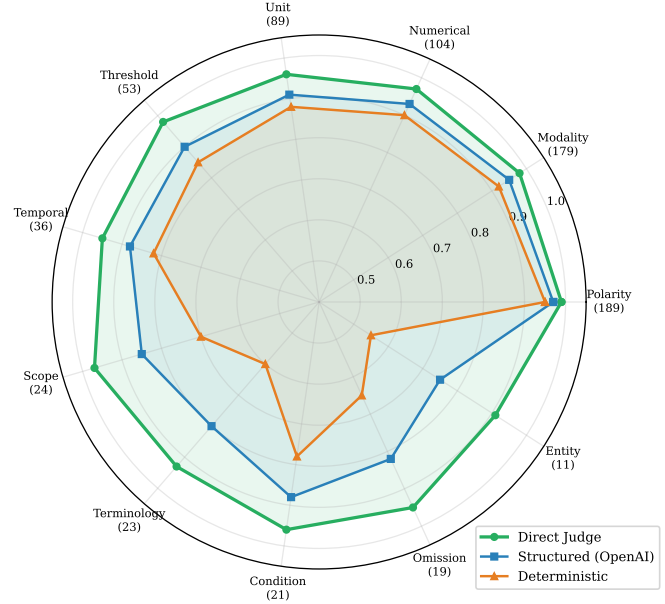


Fig. 4. Detection rate by drift type on the synthetic benchmark.

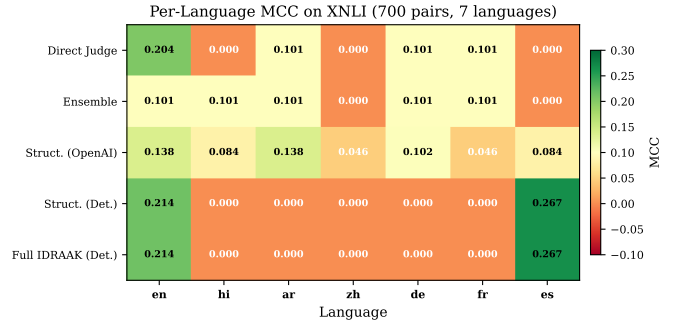


Fig. 5. Per-language MCC on XNLI across workflows.

specificity is therefore both the source of SRR’s effectiveness and its principal limitation.

D. The Hybrid Merge Strategy

Prioritizing deterministic values over LLM outputs for comparison-critical fields improved structured single MCC from 0.277 to 0.501. Deterministic extraction is reliable for numerical constraints, modality, polarity, and units, while LLM extraction provides broader coverage for entities, relations, and qualifiers. Selectively combining both approaches therefore balances precision with semantic coverage.

E. Calibration for Safety-Critical Deployment

Raw confidence scores exhibit ECE values up to 0.464, while Platt scaling reduces ECE below 0.02 across workflows. This matters in safety-critical review, where confidence thresholds may determine human escalation [28]. These results

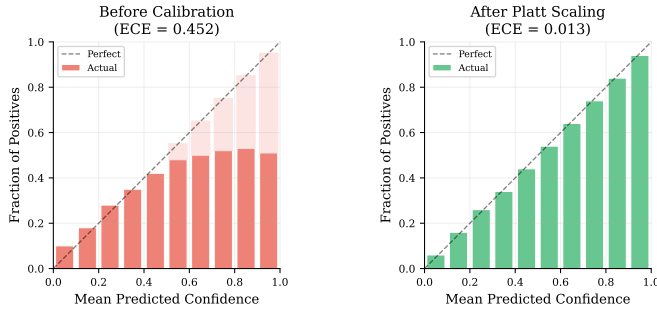


Fig. 6. Reliability diagrams before and after Platt scaling.

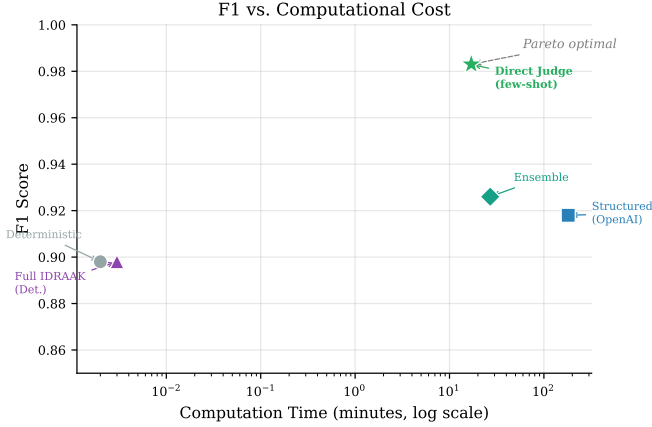


Fig. 7. F1 score vs. computation time on the synthetic benchmark.

suggest that calibration should be treated separately from classification performance when deploying LLM-based drift detectors.

F. Cross-Lingual Consistency

Across seven XNLI languages (en, hi, ar, zh, de, fr, es), LLM workflows show relatively consistent behavior, with per-language MCC varying by less than 0.1. These results indicate cross-lingual consistency rather than strong XNLI performance, since entailment is directional whereas semantic equivalence is bilateral. Deterministic extraction varies more because its patterns remain primarily English-oriented.

G. Limitations

The primary benchmark uses template-generated requirements and controlled perturbations that may not capture complex industrial translations or interacting errors. PAWSX and XNLI provide independent stress tests but are not technical-requirement corpora. Experiments use only GPT-4o-mini, and approximately 200 ensemble samples used deterministic fallback because of API rate limits. A human baseline with inter-annotator agreement is also absent, while deterministic extraction remains primarily English-oriented.

H. Threats to Validity

Internal validity. Synthetic generation and evaluation introduce potential circularity, partially mitigated through

independent PAWSX and XNLI evaluation. **External validity.** Synthetic requirements may not capture industrial jargon, cross-references, or simultaneous translation errors. **Construct validity.** MCC is robust to imbalance [36], although the 5.3:1 class ratio remains relevant, and XNLI entailment only approximates semantic equivalence. **Conclusion validity.** Fixed seeds improve reproducibility, but LLM outputs may vary across API versions; results therefore characterize the evaluated configurations rather than establishing universal superiority of few-shot prompting.

VI. CONCLUSION

IDRAAK is presented as an interpretable framework for detecting semantic drift in multilingual technical requirements. A systematic comparison of detection workflows on 890 synthetic perturbations across 10 engineering domains and two established cross-lingual benchmarks demonstrates that a single LLM call with six few-shot examples achieves the strongest performance on the technical benchmark (MCC=0.888), outperforming the evaluated structured and multi-stage configurations. The results indicate that increased architectural complexity does not necessarily improve semantic-drift detection and highlight the effectiveness of simple, well-designed prompting for this task. The Semantic Requirement Representation (SRR) complements this approach by providing structured, field-level evidence that identifies changes in technical attributes, thereby supporting interpretable decisions in safety-critical review workflows.

Further evaluation demonstrates that combining deterministic evidence with LLM judgment can be beneficial for adversarial inputs, with the ensemble achieving MCC=0.459 on PAWS-X. The hybrid extraction merge strategy is also shown to substantially affect detection quality, while post-hoc Platt scaling reduces Expected Calibration Error (ECE) from 0.452 to 0.013. Future work will focus on evaluating IDRAAK using real-world industrial requirement corpora with human-annotated drift labels, extending the analysis to additional LLM providers, and investigating lightweight multi-agent debate strategies. Expanding deterministic extraction with multilingual technical vocabularies may further improve structured analysis across languages. Finally, integrating IDRAAK into continuous integration pipelines represents a practical direction for automated semantic-faithfulness checking in multilingual specification workflows.

REFERENCES

- [1] K. Pohl, *Requirements Engineering: Fundamentals, Principles, and Techniques*. Berlin, Heidelberg: Springer, 2010.
- [2] D. M. Berry, E. Kamsties, and M. M. Krieger, "From Contract Drafting to Software Specification: Linguistic Sources of Ambiguity—A Handbook," Technical Report, University of Waterloo, 2003.
- [3] C. Rupp, *Requirements-Engineering und -Management*, 5th ed. Munich, Germany: Hanser Verlag, 2009.
- [4] IEEE, "IEEE Recommended Practice for Software Requirements Specifications," IEEE Std 830-1998, 1998.
- [5] K. Papineni, S. Roukos, T. Ward, and W.-J. Zhu, "BLEU: A method for automatic evaluation of machine translation," in *Proc. 40th Annu. Meeting Assoc. Comput. Linguistics (ACL)*, pp. 311–318, 2002.

- [6] R. Rei, C. Stewart, A. C. Farinha, and A. Lavie, "COMET: A neural framework for MT evaluation," in *Proc. 2020 Conf. Empirical Methods Natural Language Processing (EMNLP)*, pp. 2685–2702, 2020.
- [7] T. Sellam, D. Das, and A. P. Parikh, "BLEURT: Learning robust metrics for text generation," in *Proc. 58th Annu. Meeting Assoc. Comput. Linguistics (ACL)*, pp. 7881–7892, 2020.
- [8] T. Zhang, V. Kishore, F. Wu, K. Q. Weinberger, and Y. Artzi, "BERTScore: Evaluating text generation with BERT," in *Proc. 8th Int. Conf. Learning Representations (ICLR)*, 2020.
- [9] M. Freitag, R. Rei, N. Mathur, C.-K. Lo, C. Stewart, E. Avramidis, T. Kocmi, G. Foster, A. Lavie, and A. Burchardt, "Results of WMT22 metrics shared task: Stop using BLEU—Neural metrics are better and more robust," in *Proc. 7th Conf. Machine Translation (WMT)*, pp. 46–68, 2022.
- [10] E. Agirre, M. Diab, D. Cer, and A. Gonzalez-Agirre, "SemEval-2012 Task 6: A pilot on semantic textual similarity," in *Proc. 1st Joint Conf. Lexical Comput. Semantics (*SEM)*, pp. 385–393, 2012.
- [11] D. Cer, M. Diab, E. Agirre, I. Lopez-Gazpio, and L. Specia, "SemEval-2017 Task 1: Semantic textual similarity—Multilingual and cross-lingual focused evaluation," in *Proc. 11th Int. Workshop Semantic Evaluation (SemEval-2017)*, pp. 1–14, 2017.
- [12] Y. Yang, Y. Zhang, C. Tar, and J. Baldridge, "PAWS-X: A cross-lingual adversarial dataset for paraphrase identification," in *Proc. 2019 Conf. Empirical Methods Natural Language Processing (EMNLP)*, pp. 3687–3692, 2019.
- [13] A. Conneau, R. Rinott, G. Lample, A. Williams, S. Bowman, H. Schwenk, and V. Stoyanov, "XNLI: Evaluating cross-lingual sentence representations," in *Proc. 2018 Conf. Empirical Methods Natural Language Processing (EMNLP)*, pp. 2475–2485, 2018.
- [14] Y. Zhang, J. Baldridge, and L. He, "PAWS: Paraphrase adversaries from word scrambling," in *Proc. 2019 Conf. North Amer. Chapter Assoc. Comput. Linguistics (NAACL)*, pp. 1298–1308, 2019.
- [15] T. Brown, B. Mann, N. Ryder, M. Subbiah, J. D. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, et al., "Language models are few-shot learners," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, pp. 1877–1901, 2020.
- [16] OpenAI, "GPT-4 technical report," arXiv:2303.08774, 2023.
- [17] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. Chi, Q. V. Le, and D. Zhou, "Chain-of-thought prompting elicits reasoning in large language models," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35, pp. 24824–24837, 2022.
- [18] H. Touvron, L. Martin, K. Stone, P. Albert, A. Almahairi, Y. Babaei, N. Bashlykov, S. Batra, P. Bhargava, S. Bhosale, et al., "Llama 2: Open foundation and fine-tuned chat models," arXiv:2307.09288, 2023.
- [19] Q. Wu, G. Bansal, J. Zhang, Y. Wu, B. Li, E. Zhu, C. Liu, A. H. Awadallah, R. W. White, D. Burger, and A. Ahmed, "AutoGen: Enabling next-gen LLM applications via multi-agent conversation," arXiv:2308.08155, 2023.
- [20] S. Hong, M. Zhuge, J. Chen, X. Zheng, Y. Cheng, C. Zhang, J. Wang, Z. Wang, S. K. S. Yau, Z. Lin, et al., "MetaGPT: Meta programming for a multi-agent collaborative framework," in *Proc. 12th Int. Conf. Learning Representations (ICLR)*, 2024.
- [21] Y. Du, S. Li, A. Torralba, J. B. Tenenbaum, and I. Mordatch, "Improving factuality and reasoning in language models through multiagent debate," in *Proc. 41st Int. Conf. Machine Learning (ICML)*, 2024.
- [22] C.-M. Chan, W. Chen, Y. Su, J. Yu, W. Xue, S. Zhang, J. Fu, and Z. Liu, "ChatEval: Towards better LLM-based evaluators through multi-agent debate," in *Proc. 12th Int. Conf. Learning Representations (ICLR)*, 2024.
- [23] J. Liu, D. Shen, Y. Zhang, B. Dolan, L. Carin, and W. Chen, "What makes good in-context examples for GPT-3?," in *Proc. Deep Learning Inside Out (DeeLIO), ACL Workshop*, pp. 100–114, 2022.
- [24] S. Min, X. Lyu, A. Holtzman, M. Artetxe, M. Lewis, H. Hajishirzi, and L. Zettlemoyer, "Rethinking the role of demonstrations: What makes in-context learning work?," in *Proc. 2022 Conf. Empirical Methods Natural Language Processing (EMNLP)*, pp. 11048–11064, 2022.
- [25] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, "On calibration of modern neural networks," in *Proc. 34th Int. Conf. Machine Learning (ICML)*, pp. 1321–1330, 2017.
- [26] J. Platt, "Probabilistic outputs for support vector machines and comparisons to regularized likelihood methods," in *Advances in Large Margin Classifiers*. MIT Press, pp. 61–74, 1999.
- [27] B. Zadrozny and C. Elkan, "Transforming classifier scores into accurate multiclass probability estimates," in *Proc. 8th ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, pp. 694–699, 2002.
- [28] S. Kadavath, T. Conerly, A. Askell, T. Henighan, D. Drain, E. Perez, S. Schiefer, Z. Hatfield-Dodds, N. DasSarma, E. Tran-Kinnear, et al., "Language models (mostly) know what they know," arXiv:2207.05221, 2022.
- [29] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in *Proc. 2019 Conf. Empirical Methods Natural Language Processing (EMNLP)*, pp. 3982–3992, 2019.
- [30] A. Conneau, K. Khandelwal, N. Goyal, V. Chaudhary, G. Wenzek, F. Guzmán, E. Grave, M. Ott, L. Zettlemoyer, and V. Stoyanov, "Unsupervised cross-lingual representation learning at scale," in *Proc. 58th Annu. Meeting Assoc. Comput. Linguistics (ACL)*, pp. 8440–8451, 2020.
- [31] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proc. 2019 Conf. North Amer. Chapter Assoc. Comput. Linguistics (NAACL)*, pp. 4171–4186, 2019.
- [32] A. Ferrari, G. O. Spagnolo, and S. Gnesi, "Pure and cross-domain effects in NLP-based requirements engineering tasks," in *Proc. IEEE 25th Int. Requirements Engineering Conf. (RE)*, pp. 344–353, 2017.
- [33] L. Zhao, W. Alhoshan, A. Ferrari, K. J. Letsholo, M. A. Ajagbe, E.-V. Chioasca, and R. T. Batista-Navarro, "Natural language processing for requirements engineering: A systematic mapping study," *ACM Computing Surveys*, vol. 54, no. 3, pp. 1–41, 2021.
- [34] F. Dapiaz, I. van der Schalk, and S. Brinkkemper, "Detecting terminological ambiguity in user stories: Tool and experimentation," *Information and Software Technology*, vol. 110, pp. 3–16, 2019.
- [35] B. W. Matthews, "Comparison of the predicted and observed secondary structure of T4 phage lysozyme," *Biochimica et Biophysica Acta (BBA)—Protein Structure*, vol. 405, no. 2, pp. 442–451, 1975.
- [36] D. Chicco and G. Jurman, "The advantages of the Matthews correlation coefficient (MCC) over F1 score and accuracy in binary classification evaluation," *BMC Genomics*, vol. 21, no. 6, pp. 1–13, 2020.
- [37] T. Pires, E. Schlinger, and D. Garrette, "How multilingual is multilingual BERT?," in *Proc. 57th Annu. Meeting Assoc. Comput. Linguistics (ACL)*, pp. 4996–5001, 2019.
- [38] S. Wu and M. Dredze, "Are all languages created equal in multilingual BERT?," in *Proc. 5th Workshop Representation Learning for NLP*, pp. 120–130, 2020.
- [39] J. Maynez, S. Narayan, B. Bohnet, and R. McDonald, "On faithfulness and factuality in abstractive summarization," in *Proc. 58th Annu. Meeting Assoc. Comput. Linguistics (ACL)*, pp. 1906–1919, 2020.
- [40] Z. Ji, N. Lee, R. Frieske, T. Yu, D. Su, Y. Xu, E. Ishii, Y. J. Bang, A. Madotto, and P. Fung, "Survey of hallucination in natural language generation," *ACM Computing Surveys*, vol. 55, no. 12, pp. 1–38, 2023.
- [41] S. Colvin, "Pydantic: Data validation using Python type annotations," 2023. [Online]. Available: <https://docs.pydantic.dev/>
- [42] S. Edunov, M. Ott, M. Auli, and D. Grangier, "Understanding back-translation at scale," in *Proc. 2018 Conf. Empirical Methods Natural Language Processing (EMNLP)*, pp. 489–500, 2018.